Consistency Models & the CAP Theorem — Lab Report

Course CSS4004 – Distributed Systems, Lab 6-7
Topic PACELC trade-offs and quorum tuning in a distributed key-value store
Student Manap Beibarys
Repository Replicated-Counter-Service (extends Assignment 2)
Date 10 November 2026

The Assignment 2 replica process now also hosts a replicated key-value store (`kvstore.proto`, exactly the messages from Task 1 plus an internal `ReplicaPeer` service). Node A (port 50051) holds the lease for range `[a-z]`; B (50052) and C (50053) are followers. A and B form DC-1, C forms DC-2. Partitions are produced by socket interception: every replica-to-replica link runs through a TCP proxy (`netsim.py`) that the test can cut or delay. Final results: **tests/test_cap.py 17 passed in 23.56s**; whole suite including Assignment 2 tests **38 passed in 79.40s (0:01:19)** (`logs/test_cap_output.txt`, `logs/pytest_output_lab67.txt`).

Implementation summary

Request	Behaviour (server.py, class KVNode)
`Get LINEARIZABLE`	Served only by the leaseholder; a follower proxies the request to A. A answers only while its lease is valid, i.e. a majority (itself + 1) acknowledged a heartbeat within `--lease-ms` = 1000 ms. Otherwise `UNAVAILABLE` "ReplicaUnavailableError".
`Get FOLLOWER_STALE`	Served from the local replica (`is_stale = true` on a follower). Every 100 ms A sends each follower a heartbeat with a *closed timestamp* T and any CP writes the follower missed; afterwards the follower holds every committed CP write with timestamp ≤ T. If `now − T > max_staleness_ms` the read fails with `FAILED_PRECONDITION` "STALE_READ_EXCEEDED". `read_timestamp` = T.
`Put QUORUM_CP`	Forwarded to the leaseholder, which sends the version to all peers in parallel and commits only with ≥ N/2+1 = 2 acks. Without a majority nothing is applied and the reply is `success=false`, "ReplicaUnavailableError: quorum lost". A follower that cannot reach A rejects the write the same way.
`Put ALWAYS_AP`	Applied on the receiving node at once and pushed to every peer it can reach; always `success=true`, `acks_received` = replicas holding the value.
Conflict resolution	Each value carries a version (timestamp in µs, origin node); the client may set `timestamp`, otherwise a hybrid wall clock assigns it. Replicas keep the larger version (Last-Write-Wins, origin breaks ties). Anti-entropy (every 500 ms) exchanges full state with each peer, so divergent AP writes converge after the partition heals.
Retries	`idempotency_key` is deduplicated on the coordinator; a retry gets the stored reply.

Section 1: Test Results & Quorum Verification

1.1 Environment

Machine	12th Gen Intel(R) Core(TM) i7-12700, 10 logical CPUs, 19 GB RAM; client and replicas on the same host over localhost
OS	Ubuntu 22.04.5 LTS, Linux kernel 6.8.0-1070-azure, x86_64
Python	3.10.12, virtual environment .venv
Packages	grpcio 1.85.1, protobuf 7.36.2, pytest 9.2.0; stubs generated with grpcio-tools 1.80.0 (libprotoc 31.1)
Commands	python3 -m pytest tests/test_cap.py -v; python3 -m pytest -v; python3 perf_cap_bench.py

1.2 Quorum verification table (R + W vs N, N = 3)

Tests Q01–Q06 use Dynamo-style client quorums (`client.QuorumClient`) on replicas started with anti-entropy off, so a version exists only on the replicas the client wrote. For every possible write set of size W and read set of size R: write *old* to all three replicas, write *new* (larger timestamp) to the W replicas, read the R replicas and keep the newest version. Source: `logs/test_cap/quorum_matrix.md`.

W	R	R+W	vs N	combinations	fresh reads	stale reads	stale exactly when write and read sets are disjoint
2	2	4	**> 3**	9	9	0	yes
3	1	4	**> 3**	3	3	0	yes
1	3	4	**> 3**	3	3	0	yes
1	1	2	**≤ 3**	9	3	6	yes
1	2	3	**≤ 3**	9	6	3	yes
2	1	3	**≤ 3**	9	6	3	yes

With R+W>N no combination returned a stale value; with R+W≤N every disjoint combination (and only those) returned *old*. For W=R=1, 6 of 9 reads were stale, exactly the 3×2 pairs of different replicas.

1.3 Test cases (tests/test_cap.py, 17 pytest cases)

Every case starts its own three-process cluster; replica logs are kept in `logs/test_cap/<test name>/`. "DC-1 | DC-2" means all links between {A, B} and {C} are cut. The file was run 6 times during development and in the final full-suite run, with every case passing each time.

ID	Test / scenario	Expected	Actual	Result
Q01-03	`test_quorum_overlap_read_always_sees_latest_write[W-R]` (W,R) = (2,2), (3,1), (1,3); every write set × read set	R+W>N: every read returns the latest write	15/15 reads fresh	**PASS**
Q04-06	`test_quorum_without_overlap_allows_stale_read[W-R]` (W,R) = (1,1), (1,2), (2,1); every write set × read set	R+W≤N: stale reads exist; stale ⇔ sets disjoint	12/27 stale, all on disjoint sets	**PASS**
P01	`test_cp_write_on_majority_partition_succeeds` DC-1 | DC-2; QUORUM_CP Put to A, then to B (forwarded to A)	success, acks_received = 2	success, acks = 2 (both); C has no value	**PASS**
P02	`test_cp_write_on_minority_partition_rejected` DC-1 | DC-2; QUORUM_CP Put to C	failure, ReplicaUnavailableError	success=false, acks = 0, "quorum lost"; not applied anywhere, also after heal	**PASS**
P03	`test_ap_write_on_minority_partition_accepted` DC-1 | DC-2; ALWAYS_AP Put to C	success = true, acks_received = 1	success, acks = 1; only C holds the value	**PASS**
P04	`test_partition_recovery_merges_divergent_values_by_lww` AP writes to the same keys on both sides, then heal	all replicas converge to the larger timestamp	all 3 converged; newer ts wins even when written earlier	**PASS**
P05	`test_follower_read_is_local_and_fresh_when_connected` no partition; FOLLOWER_STALE and LINEARIZABLE Get on C	stale read served locally (is_stale), linearizable via A	value 10 both; is_stale = true / false	**PASS**
P06	`test_minority_refuses_linearizable_read_and_bounds_follower_read` DC-1 | DC-2; Gets on C after a CP write on A	LINEARIZABLE rejected; FOLLOWER_STALE returns old value until the bound	ReplicaUnavailableError; old value 100 (is_stale); STALE_READ_EXCEEDED at 300 ms bound	**PASS**
P07	`test_heartbeat_delivers_missed_cp_writes_within_staleness_bound` anti-entropy off; CP write during partition, wait 1.2 s, heal	first successful 1000 ms-bounded read on C includes the write	first read = v2	**PASS**
P08	`test_isolated_leaseholder_loses_lease_no_split_brain` A | B, C (leaseholder in the minority)	lease expires; no side commits CP writes; AP still accepted	lease expired, CP rejected on A and on B, AP acks 1 / 2	**PASS**
P09	`test_cp_write_survives_one_crash_not_two` SIGKILL C, CP Put; SIGKILL B, CP Put	acks = 2 then quorum lost	acks = 2; then "quorum lost: 1/3"	**PASS**
P10	`test_put_retry_with_same_idempotency_key_applied_once` Put(key K) via B, newer Put, retry of K via C	retry answered from dedup store, newer value kept	same reply, value stays v2	**PASS**
P11	`test_invalid_requests_rejected` key outside [a-z], empty key	INVALID_ARGUMENT	INVALID_ARGUMENT	**PASS**

Evidence excerpt (replica C, test P04, `logs/test_cap/test_partition_recovery_merges_divergent_values_by_lww/replica-C.log`): C accepts its AP writes with 1/3 acks during the partition; the first anti-entropy round after healing replaces its `profile` by A's version, which has the larger timestamp:

```
C: AP put cart='dc2-version' accepted (ts=1794303361697310), acks 1/3
C: AP put profile='dc2-version' accepted (ts=1794303361697810), acks 1/3
C: LWW merge from B: profile='dc1-version' (ts=1794303361698310, origin=A)
```

Section 2: Performance Evaluation (PACELC)

`perf_cap_bench.py`: 1000 measured Gets per scenario after 100 warm-up Gets, 3 repeats on fresh clusters (mean ± standard deviation over repeats). The client is in DC-2 and sends every Get to its local replica C. Scenario 1 (LINEARIZABLE) is proxied by C to the remote leaseholder A; Scenario 2 (FOLLOWER_STALE, `max_staleness_ms = 5000`) is answered by C. Raw data: `logs/cap_bench_results.md` / `.json`.

2.1 PACELC table — 3 replicas on localhost (required table)

Read Mode	Target Node	Median Latency (ms)	p95 Latency (ms)	Read Availability (%)
Linearizable	Leaseholder A (remote, proxied by gateway C)	0.99 ± 0.01	1.13 ± 0.02	100.0
Follower Stale	Local follower C	0.45 ± 0.01	0.53 ± 0.01	100.0

2.2 Same cluster with a simulated WAN between DC-1 and DC-2 (+10 ms one way)

Read Mode	Target Node	Median Latency (ms)	p95 Latency (ms)	Read Availability (%)
Linearizable	Leaseholder A (remote, proxied by gateway C)	21.84 ± 0.18	22.84 ± 0.27	100.0
Follower Stale	Local follower C	0.46 ± 0.01	0.54 ± 0.03	100.0

2.3 Partition DC-1 | DC-2, client on the minority side

Read Mode	Target Node	Median (ms)	p95 (ms)	Availability (%)	Median time to error (ms)	Errors
Linearizable	Leaseholder A (remote)	–	–	**0.0**	0.57	ReplicaUnavailableError × 1000
Follower Stale	Local follower C	0.50	0.71	**100.0**	–	–

Sustained partition (7 s, a follower read on C every 10 ms): availability 70.1 %; the last successful read came 4.91 s after the cut and the first STALE_READ_EXCEEDED at 4.92 s. The bound is measured from C's last closed timestamp, which is up to one heartbeat (100 ms) older than the cut, so refusals start slightly before 5.00 s.

2.4 Interpretation

- **Else (no partition) → Latency vs Consistency.** On localhost a linearizable read costs one extra hop (C → A): median 0.99 ms vs 0.45 ms (×2.2), p95 1.13 vs 0.53 ms. With a 20 ms inter-DC round trip the linearizable read pays it on every request (21.84 ms median, ×48), while the follower read stays local (0.46 ms). This is the “EL” choice: lower latency in exchange for reading data up to 5 s old.
- **Partition → Availability vs Consistency.** On the minority side linearizable reads drop to 0 % availability (they fail fast, median 0.57 ms, because the gateway cannot reach the leaseholder), while follower reads stay at 100 % until the staleness bound is reached. Bounded staleness makes the read path AP for at most `max_staleness_ms`, then it turns CP again.
- All availability figures without a partition are 100 %: on a healthy cluster both modes always answer.

Section 3: Conceptual Questions

Q1. Why does R + W > N guarantee linearizability, whereas R + W ≤ N allows stale reads?

A write is acknowledged once W replicas store it; a read asks R replicas and returns the newest version among the replies. If R + W > N, the read set and the write set of the last completed write cannot be disjoint: two subsets of N replicas whose sizes add up to more than N must share at least one replica (pigeonhole). That shared replica holds the latest acknowledged version, and because versions are totally ordered by (timestamp, origin) the reader picks it. So every read returns the most recent completed write, never an older one. If R + W ≤ N the coordinator may pick a read set completely outside the write set; all R replies then carry an older version and the read is stale. Section 1.2 shows both cases experimentally: 0 stale reads out of 15 for (2,2), (3,1), (1,3); for (1,1), (1,2), (2,1) a read was stale in exactly the disjoint combinations (12 of 27).

Strictly, overlap alone gives “a read sees the latest *completed* write”. Full linearizability also needs reads not to go backwards while a write is still in progress: either a reader writes the value it returns back to a write quorum before answering (ABD), or, as in our CP path, one node (the leaseholder) orders all writes and serves linearizable reads under a lease. With W = 2 of 3 writes coordinated by A and linearizable reads served by A, the store behaves as a single copy for CP data.

Q2. What trade-off did the system make for writes on the minority partition, QUORUM_CP vs ALWAYS_AP?

- **QUORUM_CP chose consistency over availability.** C cannot reach the leaseholder or a majority, so the write is rejected (P02: `success=false`, ReplicaUnavailableError, acks 0) and nothing is applied anywhere, even after healing. Clients in DC-2 lose write availability (and linearizable reads, Section 2.3) for the duration of the partition, but no two sides can ever commit different values: when the leaseholder itself was isolated (P08), neither side accepted CP writes, so there is no split brain. The majority side DC-1 keeps working (P01: acks 2).

- **ALWAYS_AP chose availability over consistency.** C accepts the write with only its own ack (P03: acks 1); both sides keep writing and the replicas diverge. After healing, anti-entropy resolves each key by Last-Write-Wins (P04). The price: LWW keeps one value and silently discards the other concurrent write (a lost update), and the winner is decided by timestamps, so a node with a lagging clock loses even if it wrote last in real time (the `profile` key). Where both updates must survive (a shopping cart, a set of tags) a union merge / CRDT would keep both, at the cost of exposing conflicts to the data model.
- In PACELC terms the system is **PC/EC** for QUORUM_CP + LINEARIZABLE and **PA/EL** for ALWAYS_AP + FOLLOWER_STALE, selectable per request.

Limitations

- The lease never moves: if A is lost, the range has no leaseholder until A returns (no Raft election / lease transfer). CP writes and linearizable reads are unavailable on both sides in that case (P08).
- With N = 3 a rejected CP write is never stored anywhere. With N ≥ 5 peers that acked a write that still missed the majority would keep it (there is no prepare/commit log).
- Anti-entropy sends the full state each round (fine for the lab's data size; a Merkle-tree digest would be used in practice). Mixing AP and CP writes on the same key gives up linearizability for that key.
- Partitions are injected on replica-to-replica links; client-to-replica traffic is direct (each client is inside its data centre).

Appendix: Reproduction

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python3 -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. counter.proto kvstore.proto
python3 -m pytest tests/test_cap.py -v      # Lab 6-7 tests
python3 -m pytest -v                        # all tests
python3 perf_cap_bench.py                   # Section 2 tables -> logs/cap_bench_results.md
```

Files: kvstore.proto, server.py (KVNode), client.py (KVClient, QuorumClient, CLI put/read), netsim.py (LinkProxy, Cluster), perf_cap_bench.py, tests/test_cap.py.